

동기 / 비동기, 블로킹 / 논블로킹

개발자 면접에서 자주 나오는 질문이고, 다양한 요구사항을 반영해서 설계하기 위해 알아야 하는 개념입니다.

개념 정리

동기(Synchronous), 비동기 (Asynchronous)

결과값의 처리 시점이 중요

- 동기(Synchronous)
 - 호출하는 함수(Caller)가 호출된 함수(Callee)의 최종 결과값을 받아서 처리하는 구조
 - 함수의 결과값이 전달되는 시점과 결과값을 반환하는 시점이 같다.
- 비동기(Asynchronous)
 - Caller가 Callee에게 작업을 맡기고, 그 결과를 나중에 반환받아서 사용하는 구조
 - e.g. callback함수, Java의 Future, JavaScript의 Promise 등
 - 함수의 결과값이 전달되는 시점과 결과값을 반환하는 시점이 다르다.

블로킹(Blocking), 논블로킹(Non-Blocking)

제어권의 반환 여부가 중요

- 블로킹(Blocking)
 - 호출된 함수가 작업을 모두 마칠때까지 제어권을 위임
 - 호출한 함수는 호출된 함수의 작업이 끝날때까지 블로킹 상태
- 논블로킹(Non-Blocking)
 - 호출된 함수가 작업을 완료하기 전에 바로 제어권을 리턴하는 구조.
 - 호출한 함수는 바로 다음 작업 수행 가능

4가지 조합 예시 정리 (Java)

동기, 블로킹 (Sync-Blocking)

가장 일반적으로 사용하는 코드 형태라고 이해하면 됩니다.

- Caller는 결과가 올 때까지 대기(동기)
- 호출된 함수는 동작이 끝날 때까지 제어권 가짐 (블로킹)

```
public Member findMember(Long id) {  
    Member member = memberRepository.findById(id).orElseThrow(() -> new  
    NoSuchElementException("그런 건 없어용"));
```

```
    return member;
}
```

```
RestTemplate restTemplate = new RestTemplate();

String result = restTemplate.getForObject("http://example.com/api", String.class);
```

비동기, 논블로킹 (Async-NonBlocking)

다른 작업을 호출하고, 바로 다음 작업을 이어서 진행하는 구조입니다.

- Caller는 함수 호출 후 바로 제어권을 돌려받아서 그 다음 일을 수행합니다.
- 작업이 완료되었을 때 콜백이나 이벤트를 통해서 결과값을 처리합니다.

```
public CompletableFuture<String> asyncService() {
    return CompletableFuture.supplyAsync(() -> {
        // 무시무시한 작업 등
        return "job finished";
    });
}

public void main() {
    CompletableFuture<String> future = asyncService();

    future.thenAccept(result -> {
        System.out.println("결과가 왔대요: " + result);
    });

    System.out.println("다른 작업");
}
```

동기, 논블로킹 (Sync-NonBlocking)

호출자가 제어권을 바로 돌려받지만, 결과값을 계속 물어보는 구조입니다. 이 경우를 Polling이라고 합니다.

- 잔소리를 자주 하는 상황으로 이해하면 이해하기 편합니다
 - (10분 전) 과제 다 했나요?
 - (5분 전) 과제 다 했나요?
 - (지금) 과제 다 했나요?

```
Future<String> future = executor.submit(() -> {
    Thread.sleep(3000);
    return "job finished";
})
```

```
while (!future.isDone()) {
    System.out.println("언제 끝남?");
    Thread.sleep(500); // 다른 작업 수행 가능
}

String result = future.get();
System.out.println("작업이 끝났네요: " + result);
```

비동기, 블로킹 (Async-Blocing)

그런데 비동기 호출을 했다면 어느 순간에는 리턴값을 사용해야 할 텐데, 이를 모두 비동기-논블로킹구조를 사용할 수 있나요?

```
// A <- e.g. DB
A(동기) - B(??) - C(비동기) - D(비동기)
```

- 이게 매우 어렵습니다.. 그래서 WebFlux 등 Spring에서도 비동기 처리를 지원하지만 적용하기가 어렵습니다. 이상적으로 비동기를 사용하기 위해서는 R2DBC 등 DB단부터 비동기 처리를 위한 설계를 해야 합니다. (JDBC는 Blocking I/O이기 때문)
- 비동기로 설계를 하고 나서 결과를 받기 위해 블로킹을 거는 형식입니다. (안티패턴)
 - e.g. API 호출 결과를 사용해야 하는 상황
- 비동기 호출 직후 블로킹을 걸게 되면 사실상 성능 이점을 보기 어렵습니다.

```
Mono<String> mono = webClient.get()
    .uri("http://example.com")
    .retrieve()
    .bodyToMono(String.class);

// 결과가 도착할때까지 멈춤
// -> 비동기의 이점을 활용하지 못함
String result = mono.block();
System.out.println(result);
```

- 비동기로 시작했다면 끝까지 비동기 기반으로 처리해야 성능상의 이점을 얻을 수 있습니다.

요약

조합	설명	Java/Spring 키워드
Sync-Blocking	결과가 올 때까지 멈춰서 기다림 (가장 일반적인 형태)	method call, JDBC, RestTemplate
Async-NonBlocking	요청을 보낸 후 바로 제어권을 돌려받아서 다른 일 수행, 결과는 콜백으로 받음 (이상적인 형태)	CompletableFuture, WebFlux(WebClient)

조합	설명	Java/Spring 키워드
Sync-NonBlocking	다른 일 하면서 중간중간 확인 (Polling)	<code>Future.isDone()</code> <code>loop</code>
Async-Blocking	비동기로 시키고 결과 나올 때까지 블로킹 (비효율)	<code>Future.get()</code> , <code>Mono.block()</code>